SPAD IN 1 SLIDE · STRUCTURED PROMPT-DRIVEN ENGINEERING

Building software with AI, under control

Think, validate, execute: a method so that the code AI writes can be reviewed, explained and audited. Illustration in Spanish.

The problem →

AI writes code very fast. Without a method, nobody decides the design, tests are missing and no evidence remains. Speed turns into debt and risk.

11 phases that block →

From context (phase 0) to plan, tests, code, reviews and version (phase 10). Without validation, no progress. Small work: reduced version.

Each AI, a single role →

One AI plans, another builds and a different one reviews. The builder does not decide the design. No AI approves its own work.

GO · GO with changes · NO-GO →

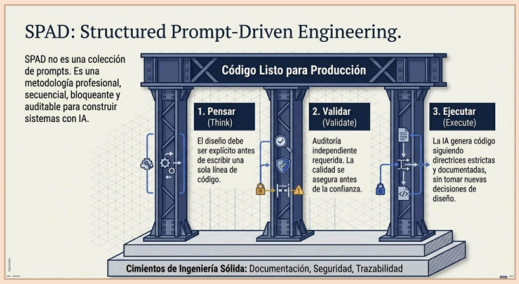
Each review gives a verdict. A person accepts it and also reviews the code. A well-founded NO-GO is the method working.

Everything is written down →

Contexts with company and project rules, one topic per piece of work, and a record of which model made each artefact and who validated it.

Fit with SEVEN-G and SPHERES →

SPHERES: where and with what ambition. SEVEN-G: what is built, with what risk and how it is measured. SPAD: how it is built (phases 4 and 5).



SPAD promises no figures and certifies nothing: it leaves evidence so that your company can measure and audit what AI builds.

→ What SPAD is · → Operating guide: phases and roles · → SEVEN-G 53 · Building with AI · → What SPHERES is

SPAD · Version 0.x, under review · CC BY 4.0 · Fernando García Varela seachad.github.io/AI_CONSULTING/ 1 / 1

Notes

SPAD (Structured Prompt-Driven Engineering) is Fernando García Varela's methodology for building software with the help of artificial intelligence in a governed way. It is not a collection of instructions for requesting code: it is a way of working that is sequential, blocking and auditable. The illustration sums it up in three pillars: think (the design is explicit before a single line of code is written), validate (an independent review) and execute (the AI writes the code without taking design decisions).

The problem. AI generates plausible code at great speed. Without structure, that speed produces decisions that nobody has taken consciously, scarce tests, vulnerabilities that nobody sees and technical debt. Example: a team requests code until the demonstration works; in production it fails and nobody knows why the system is built the way it is. Why it matters: the problem is not AI, it is the lack of method, and AI amplifies it.

The phases. The main cycle has eleven phases, from 0 to 10: context and objective; plan, with no code; plan review; code primer; test strategy; implementation; test implementation; test review; code review; minimal fixes; and version management, which is the handover to operations. No phase starts until the previous one has left its artefact and a person has validated it. Why it matters: an architecture error detected in the plan review costs a conversation; detected in production, it costs an incident.

The roles. A person orchestrates: sets the objective, validates each output and has the final say. The AI planner designs and does not write code. The AI builder writes the code and does not decide the design. The AI reviewer, different from the one that builds, evaluates and does not fix. The AI fixer applies minimal fixes. It is the segregation of duties of any serious process: whoever does the work does not approve it. The AI reviewer is not the SEVEN-G AI Auditor, who is a person.

The verdicts. Each review ends with GO (work advances), GO with changes (work is adjusted and reviewed again) or NO-GO (work goes back). A GO with serious open problems is a self-approval and is discarded. In addition, a competent person reviews the code before it is merged; for sensitive code (payments, personal data, action limits of agents), a second person does. The validation policy adds a rule: a response that skips its phase is discarded in its entirety, even if the code works. A valid NO-GO is worth more than an invalid GO.

Contexts and records. Before working, two contexts are loaded: the global one, with the company's rules (for example, no real personal data is sent to an external AI provider), and the project one. Each piece of work has a topic, for example pdf_export, that groups all its artefacts. Each artefact records which model produced it and which person validated it. Why it matters: months later, anyone can reconstruct why something was built that way, and an audit finds the evidence in a single place.

Reduced version. For limited changes, with no personal data or money, the reduced version (SPAD Lite) groups the phases into three artefacts: reviewed plan, reviewed delivery and version. The separation between the AI that builds and the one that reviews is not cut, nor is human validation. If the software contains AI, SPAD adds an evaluation of the system's behaviour.

Fit with the other frameworks. SPHERES helps decide where AI adds value (nine spheres) and with what ambition (Optimise, Augment, Transform). SEVEN-G decides which initiative is built, with what risk and how its value is measured. SPAD governs how the software is built, mainly in SEVEN-G phases 4 (Solution design) and 5 (Delivery and validation) (document 53). SPAD verdicts do not replace SEVEN-G decision gates: its artefacts are inputs to the evidence. SEVEN-G can be used with SPAD or with another equivalent documented method.

What SPAD does not do: it promises no improvement figures (each organisation measures its own with the process metrics of document 08) and it certifies nothing; the self-assessment of conformity is carried out by the organisation itself.

Question for the classroom: if your team uses AI to write code, who decides the design today and who reviews what the AI writes?

SOURCES AND LINKS

- **SPAD 00 · What SPAD is and what it is for** https://seachad.github.io/AI_CONSULTING/SPAD/html/en/00_SPAD_Que_es_y_para_que_sirve.html
- **SPAD 01 · Operating guide (phases, roles, verdicts, reduced version)** https://seachad.github.io/AI_CONSULTING/SPAD/html/en/01_SPAD_Guia_operativa.html
- **SPAD 02 · Contexts, topics and artefact records** https://seachad.github.io/AI_CONSULTING/SPAD/html/en/02_SPAD_Contextos_temas_y_registro.html
- **SPAD 03 · Validation policy** https://seachad.github.io/AI_CONSULTING/SPAD/html/en/03_SPAD_Politica_de_validacion.html
- **SEVEN-G 53 · Building solutions with AI** https://seachad.github.io/AI_CONSULTING/SEVEN-G/html/en/53_SEVEN-G_Construccion_de_soluciones_con_IA.html#relationship-between-seven-g-and-spad
- **SPHERES 00 · What SPHERES is and what it is for** https://seachad.github.io/AI_CONSULTING/SPHERES/html/en/00_SPHERES_Que_es_y_para_que_sirve.html
- **What SPAD is** https://seachad.github.io/AI_CONSULTING/SPAD/html/en/00_SPAD_Que_es_y_para_que_sirve.html
- **Operating guide: phases and roles** https://seachad.github.io/AI_CONSULTING/SPAD/html/en/01_SPAD_Guia_operativa.html
- **SEVEN-G 53 · Building with AI** https://seachad.github.io/AI_CONSULTING/SEVEN-G/html/en/53_SEVEN-G_Construccion_de_soluciones_con_IA.html
- **What SPHERES is** https://seachad.github.io/AI_CONSULTING/SPHERES/html/en/00_SPHERES_Que_es_y_para_que_sirve.html

LEGAL NOTICE

Legal notice: a generic, free methodological aid provided "as is" for information purposes; it is not legal advice and has not been legally reviewed for any case. Each organisation adapts it and is responsible for its legal review and its regulatory compliance; it may not reflect recent regulatory changes. To the fullest extent permitted by law, the author accepts no liability whatsoever for the effects of its application. It does not grant certification of any kind. Details in document 93.

CC BY 4.0 (content) · MIT (code) · Fernando García Varela · seachad.github.io/AI_CONSULTING/